Day2-Nginx-Webserver

🎯 **Goal:** Practice a production-style investigation when an Nginx website on an Ubuntu EC2 instance suddenly stops responding.
What you will learn: Verify service health, confirm port listening, check logs, validate network controls, and restore service safely.

Scenario

A website hosted on an **AWS EC2 (Ubuntu)** instance suddenly becomes unreachable.

Symptoms (examples):

- "This site can't be reached"
- "Connection refused"

Common real-world causes:

- Nginx service stopped or crashed
- Port **80** not listening
- Security group or firewall (UFW/iptables) blocking traffic
- Incorrect Nginx configuration
- Upstream application failure (if using reverse proxy)

Architecture (high level)

- User browser → HTTP → EC2 (Ubuntu) → Nginx

Tools used

- `systemctl` (service lifecycle)
- `ps` (process verification)
- `ss` (ports and sockets)

- `curl` (local HTTP test)
- `journalctl` (service logs)

Step 1 — Install Nginx

SSH into your EC2 instance and run:

```
sudo apt update
sudo apt install nginx -y
```

Verify that Nginx is running:

```
systemctl status nginx
```

Expected: **active (running)**

```
[ec2-user@ip-172-31-26-97 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
     Active: active (running) since Sat 2026-03-07 07:08:36 UTC; 20s ago
    Process: 25693 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
    Process: 25701 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
    Process: 25713 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 25728 (nginx)
      Tasks: 3 (limit: 1067)
     Memory: 3.2M
        CPU: 83ms
     CGroup: /system.slice/nginx.service
             ├─25728 "nginx: master process /usr/sbin/nginx"
             ├─25729 "nginx: worker process"
             └─25730 "nginx: worker process"

Mar 07 07:08:36 ip-172-31-26-97.us-west-2.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Mar 07 07:08:36 ip-172-31-26-97.us-west-2.compute.internal nginx[25701]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Mar 07 07:08:36 ip-172-31-26-97.us-west-2.compute.internal nginx[25701]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Mar 07 07:08:36 ip-172-31-26-97.us-west-2.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
```

Step 2 — Validate the web server (local + external)

2.1 Test from the server

```
curl -I http://localhost
```

Expected: `HTTP/1.1 200 OK`

(Optional full page)

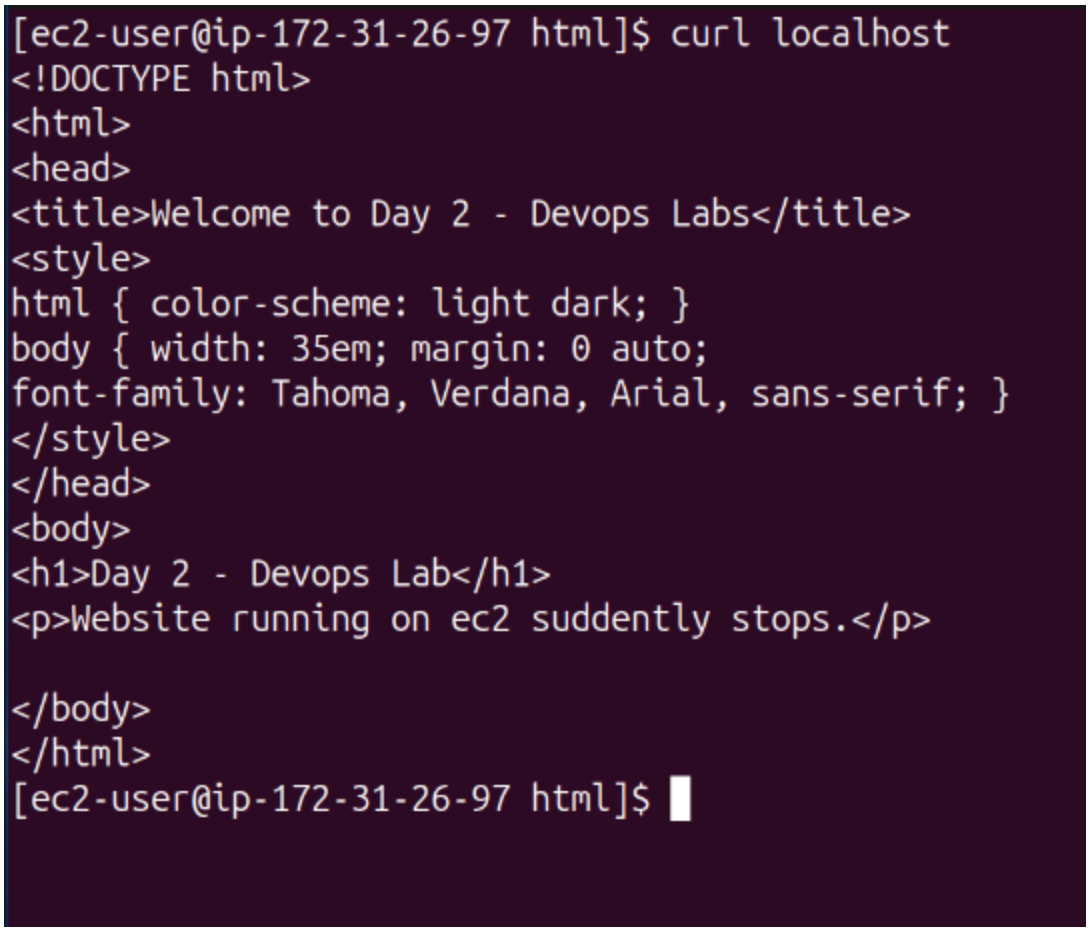
```
curl http://localhost
```

2.2 Test from your laptop browser

Open:

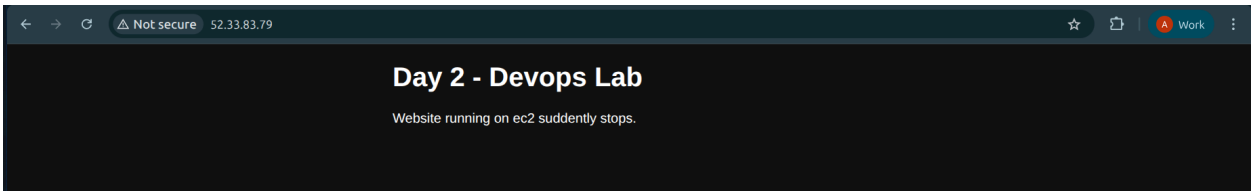
- `http://<EC2-PUBLIC-IP>`

Expected: **Nginx welcome page**



```
[ec2-user@ip-172-31-26-97 html]$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to Day 2 - Devops Labs</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Day 2 - Devops Lab</h1>
<p>Website running on ec2 suddently stops.</p>

</body>
</html>
[ec2-user@ip-172-31-26-97 html]$
```



Step 3 — Confirm port 80 is listening

```
sudo ss -tulnp | grep ':80'
```

Example:

- `LISTEN 0 511 *:80`

Meaning: Nginx is listening for HTTP traffic.

```
[ec2-user@ip-172-31-26-97 html]$ ss -tulpn
Netid       State       Recv-Q      Send-Q                                  Local Address:Port                    Peer Address:Port          Process
udp         UNCONN      0           0                                    172.31.26.97%ens5:68                        0.0.0.0:*
udp         UNCONN      0           0                                        127.0.0.1:323                           0.0.0.0:*
udp         UNCONN      0           0                        [fe80::c:ccff:fe5a:1177]%ens5:546                          [::]:*
udp         UNCONN      0           0                                          [::1]:323                             [::]:*
tcp         LISTEN      0           128                                      0.0.0.0:22                            0.0.0.0:*
tcp         LISTEN      0           511                                      0.0.0.0:80                            0.0.0.0:*
tcp         LISTEN      0           128                                         [::]:22                               [::]:*
tcp         LISTEN      0           511                                         [::]:80                               [::]:*
[ec2-user@ip-172-31-26-97 html]$
```

Step 4 — Simulate the incident (service failure)

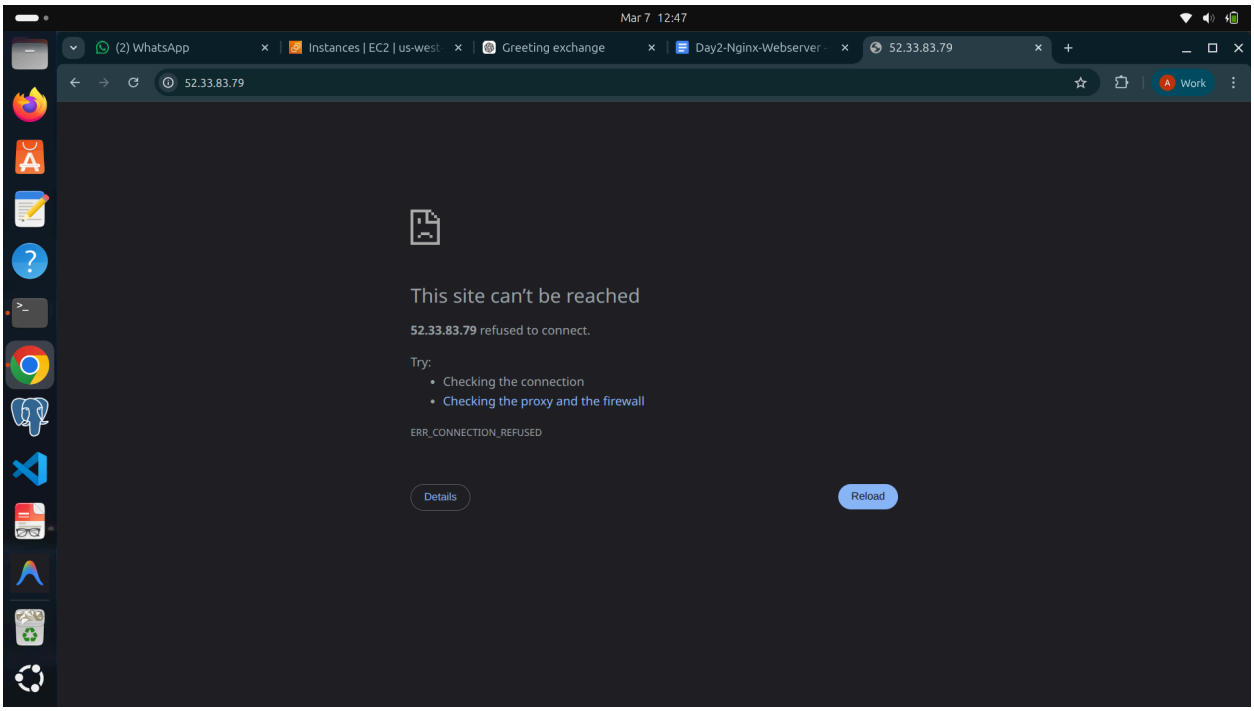
Stop Nginx:

```
sudo systemctl stop nginx
```

Now refresh:

- `http://<EC2-PUBLIC-IP>`

Expected: site becomes unreachable.



Step 5 — Investigate like a production engineer

Work from **outside-in**, then **host**, then **service**.

5.1 Quick service check

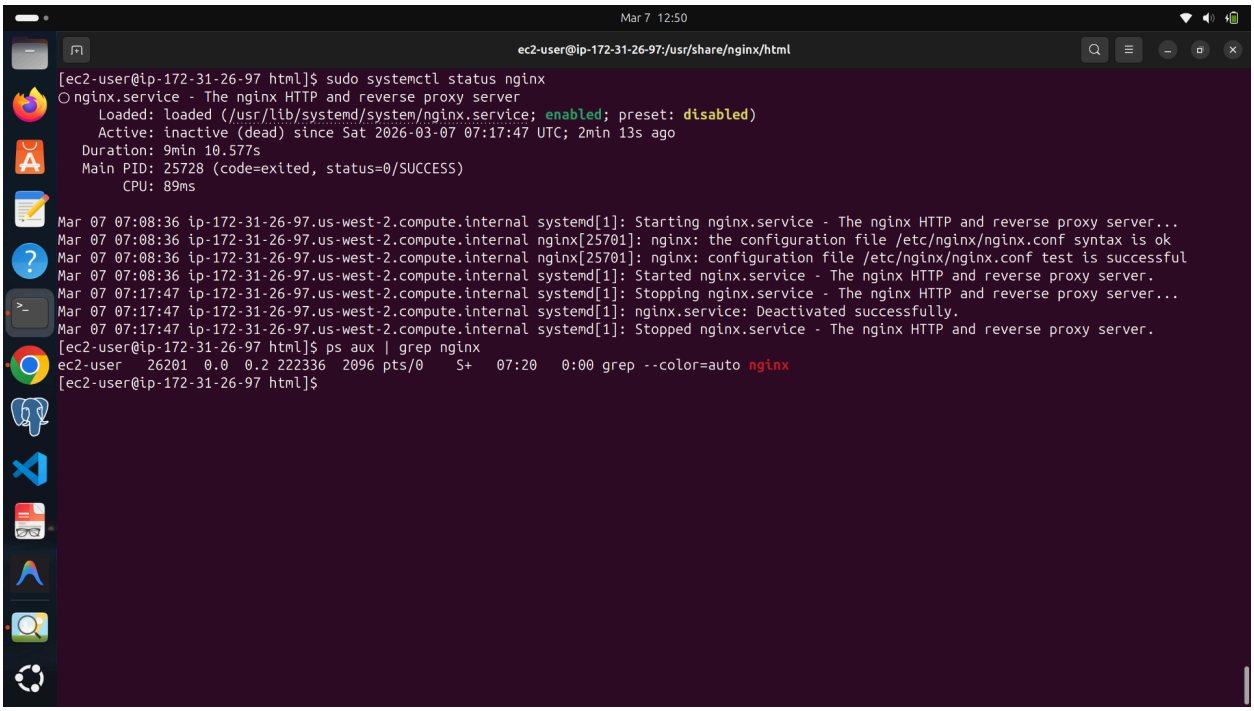
```
systemctl status nginx
```

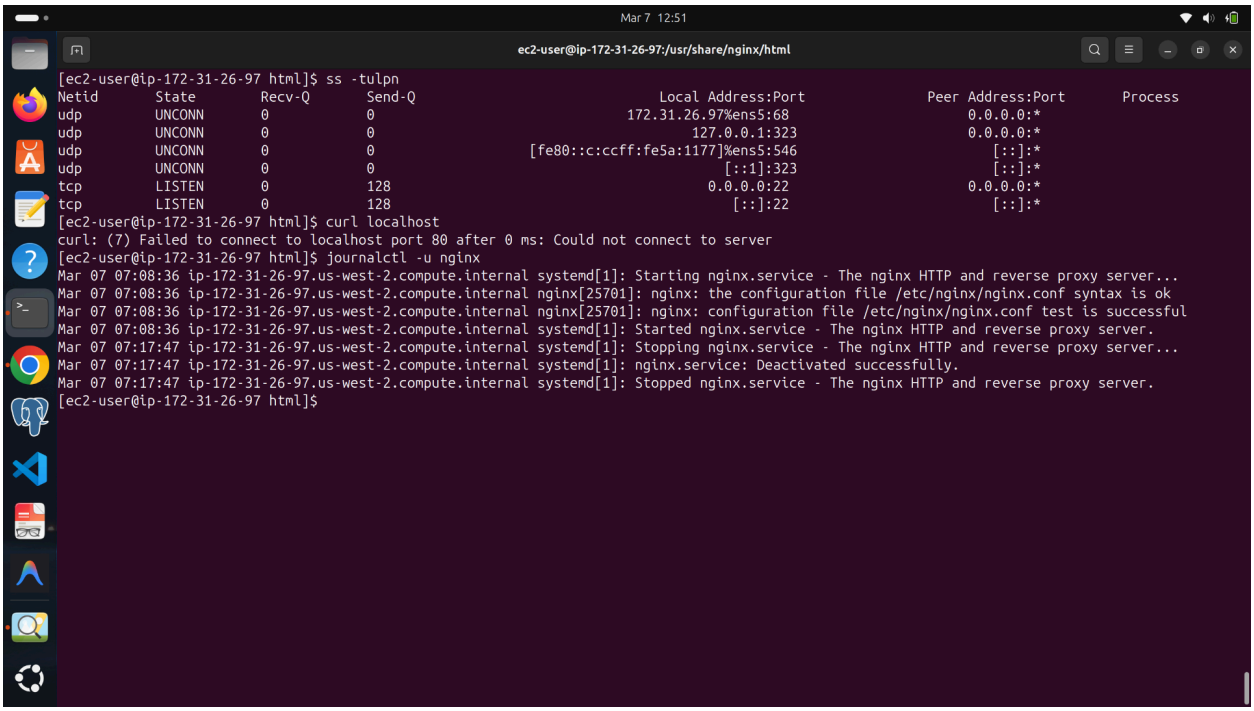
Expected (in this lab): **inactive (dead)**

5.2 Process verification

```
ps aux | grep -E 'nginx|nginx:'
```

Expected: no running Nginx master/worker processes.





5.3 Port verification

```
sudo ss -tulnp | grep ':80'
```

Expected: no listener on port 80.

5.4 Logs (what happened and why?)

```
sudo journalctl -u nginx --no-pager -n 50
```

Look for:

- crash messages
- config parse errors
- permission issues

Step 6 — Root cause (lab)

Nginx was stopped. Without a process listening on **port 80**, the server cannot respond to HTTP requests.

Step 7 — Resolution and verification

Start Nginx:

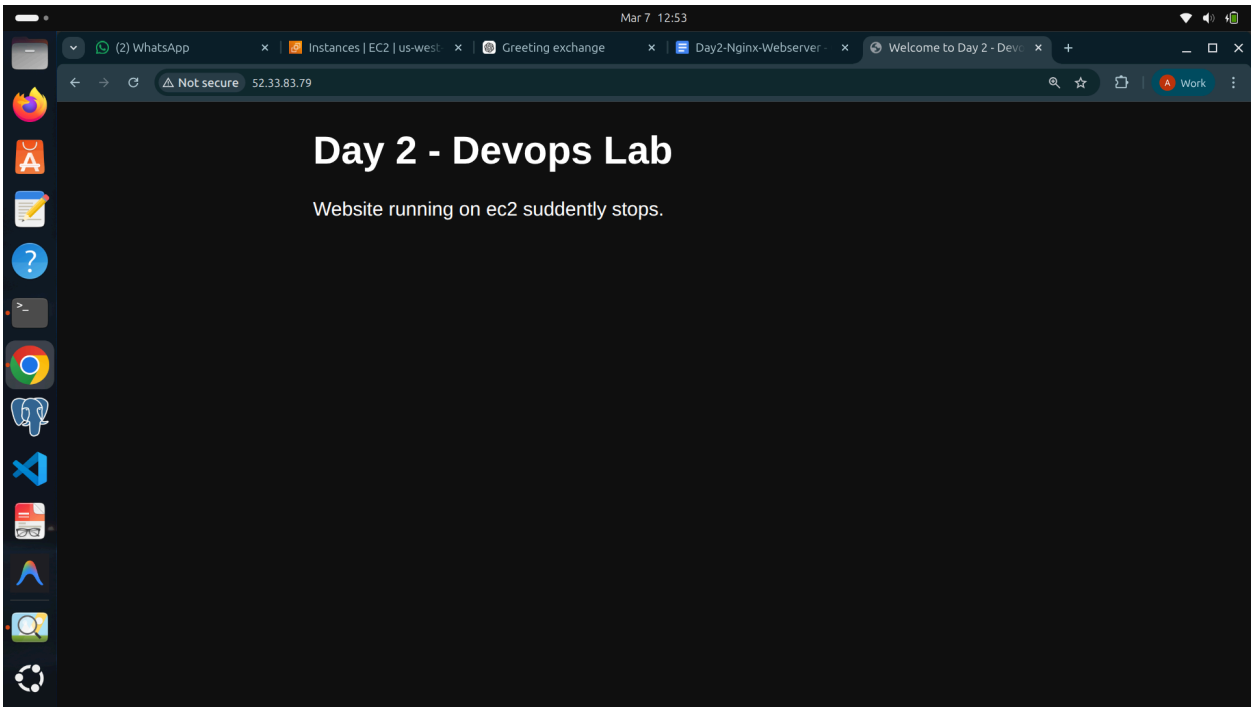
```
sudo systemctl start nginx
```

Verify:

```
systemctl status nginx
sudo ss -tulnp | grep ':80'
curl -I http://localhost
```

Then test again from your browser:

- `http://<EC2-PUBLIC-IP>`



DevOps takeaway

A reliable incident workflow:

1. Confirm the symptom (what exactly is failing?)
2. Check network path (security group, firewall)
3. Check service status
4. Confirm process and listening port
5. Check logs
6. Apply the smallest safe fix
7. Verify from local and external perspectives

Interview questions (based on this lab)

Fundamentals

1. What is the difference between **“connection refused”** and **“timeout”**?

2. What is Nginx's role in a typical web architecture?
3. What is the difference between a **service** being down vs an **application upstream** being down?

Linux and troubleshooting

1. Walk through your steps to debug a website that is down on Linux.
2. What does `systemctl status nginx` tell you, and what would you check next?
3. What is the difference between `ps` and `systemctl` for debugging?
4. How do you confirm which process is bound to port 80?
5. Where would you check Nginx logs on Ubuntu, and what would you look for?

Networking and AWS

1. How would you confirm if the issue is the EC2 security group vs the instance firewall?
2. What inbound rule is required to serve HTTP publicly?
3. If `curl localhost` works but browser access fails, what does that indicate?

Nginx operations

1. How do you validate Nginx configuration before restarting?
   - (Expected command: `nginx -t`)
2. What is the safest way to reload configuration without dropping connections?
   - (Expected: `systemctl reload nginx` or `nginx -s reload`)
3. What is the difference between `restart` and `reload`?

Incident response and reliability

1. What information would you include in an incident update to stakeholders?
2. What preventive actions would you implement so this incident is detected earlier (monitoring/alerts)?

Knowledge check (no Google)

1. Which command checks whether a service is running?
2. Which command shows listening ports and processes?
3. Which command tests a web server locally?
4. What does `journalctl -u nginx` help you investigate?
5. Why does the website stop working if nothing is listening on port 80?